

3.3.3 Przegląd algorytmów realizujących technikę tolerowania uszkodzeń

Algorytm Koo – Toueg

Koo i Toueg zaproponowali efektywny algorytm wstecznego odtwarzania stanu w technice skoordynowanego tworzenia punktu kontrolnego. Poniżej przedstawiony jest opis tego algorytmu.

Założenia.

- Procesy komunikują się tylko poprzez wymianę komunikatów przez kanały.
- Kanały są niezawodne: komunikaty nie mogą być duplikowane, tracone oraz są bezbłędne.
- Kanały zapewniają dostarczenie komunikatów z zachowania kolejności tj. są kanałami FIFO.
- Procesy zatrzymują się podczas uszkodzenia każdorazowo, gdy proces zostaje uszkodzony, wszystkie inne procesy są informowane o błędzie w skończonym czasie.
- Uszkodzenie procesu nie dzieli sieci komunikacyjnej.
- Obliczenia są w ogólności niedeterministyczne.
- W odniesieniu do spójności, wszystkie komunikaty są znaczące (tzn. powodują zmianę stanu), wszystkie gubione komunikaty są akceptowalne (jest mechanizm do odnawiania i powtórnego przesłania utraconych komunikatów) jednak w systemie nie występują komunikaty osierocone.
- Wszystkie działania mogą być przywrócone (wszystkie procesy mogą być cofnięte) tj. nie ma interakcji ze światem zewnętrznym.

Algorytm ustanawiania punktu kontrolnego oraz algorytm odtworzenia oparte są na przetwarzaniu dyfuzyjnym i dwufazowym protokole zatwierdzania (ang. *two-phase-commit protocol*).

Ze względu na uproszczenie rozważań założymy, że jest tylko jeden inicjujący kontroler, który rozpoczyna działanie algorytmu ustanawiania punktu kontrolnego oraz algorytmu wycofania do ostatniego zapamiętanego stanu. Do każdej wiadomości m wysyłanej przez proces aplikacyjny P_i , odpowiedni kontroler C_i dołącza etykietę $send_num_i$. Konkatencja komunikatu oraz etykiety będzie oznaczana przez „ $m * send_num_i$ ”. Etykiety są inkrementowane przez kontrolery przed wysłaniem komunikatu. W związku z tym, $send_num_i$ w każdej chwili reprezentuje liczbę komunikatów aplikacyjnych wysłanych od P_i do tej chwili. Zakładamy, że możliwe wartości etykiety zawierają się w przedziale od 0 do MAX.

Działanie algorytmu podczas bezbłędnej pracy.

Algorytm zapisuje dwa typy punktów kontrolnych w pamięci trwałej: punkt kontrolny trwały (ang. *permanent*) i wstępny (ang. *tentative*). Punkt kontrolny trwały jest lokalnym punktem kontrolnym procesu i jest częścią spójnej linii odzysku (ang. *recovery line*) wykorzystywanej do odtworzenia stanu. Wstępny punkt kontrolny jest tymczasowym punktem kontrolnym, który staje się trwałym punktem kontrolnym, kiedy algorytm ustanawiania globalnego spójnego punktu kontrolnego zakończy się pomyślnie. Stan początkowy wszystkich procesów jest zapisywany jako trwały punkt kontrolny, aby utworzyć początkową spójną linię odzysku. Aby zminimalizować liczbę procesów zapisujących punkt kontrolny, kontroler procesu P_i zapisuje wstępny punkt kontrolny tylko wtedy, gdy istnieje proces P_j , którego wstępny punkt kontrolny zarejestrował otrzymanie komunikatu od P_i i ostatni trwały punkt kontrolny procesu P_i nie zarejestrował wysłania tego komunikatu. Aby zaimplementować powyższą ideę, zostały wprowadzone specjalne struktury kontrolne. Dla każdych dwóch procesów P_i i P_j , niech m będzie ostatnim komunikatem takim, że proces P_i otrzymał ten komunikat od P_j po zapisaniu przez P_i trwałego lub wstępnego punktu kontrolnego. Wtedy:

$$last_rcvd_msg_i[j] = \begin{cases} send_num_j, & \text{gdy } m \text{ istnieje} \\ 0, & \text{w przeciwnym przypadku} \end{cases}$$

Niech m będzie pierwszym komunikatem, którego proces P_i wysłał do procesu P_j po zapisaniu przez P_i trwałego lub wstępnego punktu kontrolnego. Wtedy:

$$first_sent_msg_i[j] = \begin{cases} send_num_j, & \text{gdy } m \text{ istnieje} \\ 0, & \text{w przeciwnym przypadku} \end{cases}$$

Kiedy kontroler C_i żąda od kontrolera C_j zapisania wstępnego punktu kontrolnego, C_i wysyła komunikat do C_j razem z etykietą $last_rcvd_msg_i[j]$. W odpowiedzi C_j zapisuje wstępny punkt kontrolny tylko wtedy, gdy:

$$last_rcvd_msg_i[j] \geq first_sent_msg_j[i] > 0,$$

tj. kiedy istnieje komunikat zapisany przez C_i jako otrzymany, ale jeszcze nie zapisany przez C_j jako wysłany.

Zdefiniowany jest również zbiór $ckpt_cohort_i$, jako zbiór tych wszystkich kontrolerów, które powinny być zapytane o zapisanie wstępnego punktu kontrolnego, kiedy C_i zdecyduje się zapisać wstępny punkt kontrolny. Stąd:

$$ckpt_cohort_i = \{C_j: last_rcvd_msg_i[j] > 0\}$$

Zbiór ten zawiera kontrolery wszystkich procesów P_j , od których P_i otrzymał komunikat po tym jak C_i zapisał ostatni punkt kontrolny. Stąd, jeśli C_i zapisuje nowy punkt kontrolny, wtedy te kontrolery powinny również zapisać punkt kontrolny, aby zarejestrować wysłanie otrzymanego komunikatu.

Ostatecznie każdy kontroler C_i przetrzymuje zmienną $agree_to_take_ckpt_i$, aby oznaczyć swoją akceptację i możliwość do zapisania punktu kontrolnego. Kiedy tylko kontroler C_i otrzyma od jakiegokolwiek innego kontrolera żądanie:

$$m = „take_tentative_ckpt * last_rcvd_msg_j[i]”,$$

dziedziczy to żądanie, pod warunkiem że:

$$(agree_to_take_ckpt_i) \wedge (last_rcvd_msg_j[i] \geq first_sent_msg_j[j] > 0)$$

Jeśli kontroler C_i odziedziczy żądanie, zapisuje wstępny punkt kontrolny i rozpropagowuje żądanie

$$m = „take_tentative_ckpt * last_rcvd_msg_j[r]”,$$

do wszystkich kontrolerów C_r zawartych w zbiorze $ckpt_cohort_i$, następnie czeka na wszystkie odpowiedzi przenoszące wartość $agree_to_take_ckpt_r$. Po zebraniu wszystkich odpowiedzi C_i wysyła do C_j własną odpowiedź z wartością *true* lub *false* odpowiednio do rezultatu poniższego warunku:

$$agree_to_take_ckpt_i \wedge \left(\bigvee_{C_r \in ckpt_cohort_i} agree_to_take_ckpt_r \right)$$

Jeśli C_i nie odziedziczy rezultatu od C_j , odsyła natychmiast odpowiedź:

$$m = \text{„agree_to_take_ckpt}_i\text{”}$$

Aby utworzyć globalny punkt kontrolny kontroler C_α zwany inicjatorem, wysyła do wybranego kontrolera C_i żądanie:

$$m = \text{„take_tentative_ckpt * MAX”}.$$

Zauważmy, że jeśli $\text{agree_to_take_ckpt}_i$ ma wartość *true*, wtedy żądanie to jest zawsze dziedziczone.

Pierwsza faza algorytmu tworzenia punktu kontrolnego kończy się, gdy inicjator C_α zbierze wszystkie odpowiedzi. Jeśli wszystkie odpowiedzi przenoszą wartość *true*, wtedy inicjator decyduje się na utworzenie z tymczasowego punktu kontrolnego trwałego punktu kontrolnego. W przeciwnym przypadku decyduje się na wycofanie wszystkich tymczasowych punktów kontrolnych. Kiedy tymczasowy punkt kontrolny zostanie przekształcony w trwały, stary trwały punkt kontrolny może zostać odrzucony, aby odzyskać przestrzeń na trwałym nośniku. Krytyczne dla poprawności i odporności na błędy tego algorytmu jest wstrzymanie wysyłania jakichkolwiek wiadomości przez P_i , od momentu kiedy C_i zapisze wstępny punkt kontrolny do czasu otrzymania decyzji od inicjatora

Zachowanie się inicjatora C_α przedstawia blok S1. Akcje podejmowane przez kontroler C_i przedstawiają bloki S2 - S5.

S1: *when an initiator C_α spontaneously decides to take another global checkpoint, C_α will do*

```
send ( $C_\alpha$ ,  $C_i$ , „take\_tentative\_ckpt * MAX”);  
rec ( $C_\alpha$ ,  $C_i$ , „agree\_to\_take\_ckpti”);  
if agree\_to\_take\_ckpt  
  then  
    send ( $C_\alpha$ ,  $C_i$ , „make\_tentative\_ckpt\_permanent”);  
  else  
    send ( $C_\alpha$ ,  $C_i$ , „undo\_tentative\_ckpt”);
```

S2: *when P_i wants to send a message m to P_j , C_i will do*

```
wait (sending_enablei);  
send_numi[j] = send_numi[j] + 1;
```

```
    append send_numi[j] + 1;  
    send (Ci, Cj, „m * send_numi[j]”);  
S3: when Pi wants to receive a message m from Pj, Cj will do  
    rec (Ci, Cj, „m * send_numi[j]”);  
    extract send_numi[j];  
    return to Pi parameters of the receive operation: Pi, Pj and m;  
  
S4: when Ci receives from Cj message m = „take_tentative_ckpt * last_rcvd_msgj[i]”,  
Ci will do  
    if (agree_to_take_ckpti) ∧ (last_rcvd_msgj[i] ≥ first_sent_msgj[j] > 0)  
    then  
        /* disable sending of messages from Pi */  
        sending_enablei := false;  
        take tentative checkpoint of Pi;  
        for all Cr ∈ ckpt_cohorti do  
            send (Ci, Cr, „take_tentative_ckpt * last_rcvd_msgj[r]”);  
        for all Cr ∈ ckpt_cohorti do  
            rec (Ci, Cr, „agree_to_take_ckptr”);  
        agree_to_take_ckpti := (agree_to_take_ckpti) ∧  
                                (  $\bigwedge_{C_r \in \text{ckpt\_cohort}_i}$  agree_to_take_ckptr )  
        send (Ci, Cj, „agree_to_take_ckpti”);  
  
S5: when Ci receives from Cj message m, where  
m = „make_tentative_ckpt_permanent” or  
m = „undo_tentative_ckpt”, Ci will do  
    if m = „make_tentative_ckpt_permanent”  
    then  
        make tentative checkpoint permanent;  
    else  
        undo tentative checkpoint;  
    for all Cr ∈ ckpt_cohorti do  
        send (Ci, Cr, m);  
    signal (sending_enablei);
```

Działanie algorytmu podczas fazy „recovery”.

Algorytm monitoruje stan wszystkich procesów aplikacyjnych i kiedy jeden z procesów zostanie uszkodzony, inicjowana jest pierwsza faza wycofania. W fazie tej inicjator C_β wysyła do wszystkich kontrolerów żądanie do przygotowania się do restartu procesów aplikacyjnych od ich ostatniego zapisanego trwałego punktu kontrolnego, a następnie oczekuje na odpowiedzi. Kiedy C_β otrzyma wszystkie odpowiedzi pozytywne, tj. $agree_to_roll_back_i$ ma wartość *true* dla wszystkich i , C_β podejmuje decyzję do wykonania restartu. Jednakże, jeżeli jakiś proces nie zaakceptuje żądania do restartu, C_β decyduje się na kontynuowanie normalnej pracy. Decyzja inicjatora jest następnie rozpropagowywana do wszystkich kontrolerów.

Niech $roll_back_together_i$ będzie zbiorem kontrolerów C_j takich, że P_i wysłał komunikat do P_j po ostatnim utworzeniu punktu kontrolnego.

Jeżeli P_i wymaga wycofania, kontroler C_i powinien rozesłać to żądanie tylko do kontrolerów zawartych w zbiorze $roll_back_together_i$.

Niech $roll_cohort_i$ będzie zbiorem wszystkich kontrolerów C_j , takich, że proces P_i mógł wysłać komunikat do P_j , po ostatnim utworzeniu trwałego punktu kontrolnego. W rzeczywistości zbiór $roll_cohort_i$ jest zbiorem sąsiadów węzła C_j . Zachodzi relacja $roll_back_together_i \subseteq roll_cohort_i$.

Dla dowolnej pary procesów P_i i P_j niech m będzie ostatnim komunikatem, jaki proces P_i wysłał do procesu P_j po ostatnim utworzeniu trwałego punktu kontrolnego. Wtedy zdefiniujemy:

$$last_sent_msg_i[j] = \begin{cases} send_num_i, & \text{gdy } m \text{ istnieje} \\ MAX, & \text{w przeciwnym przypadku} \end{cases}$$

Dla uproszczenia założymy, że ta struktura jest kontrolowana przez C_i , ale jest częścią procesu aplikacyjnego P_i . Zauważmy, że zbiór $roll_back_together_i$ uszkodzonego procesu P_i skomponowany jest z tych kontrolerów C_j należących do $roll_cohort_i$ dla których:

$$last_rcvd_msg_j[i] > last_sent_msg_i[j]$$

Powyższy warunek może być w łatwy sposób zweryfikowany przez kontroler C_j , kiedy otrzyma od C_j komunikat kontrolny z odpowiadającą mu etykietą $last_sent_msg_i[j]$. Stąd, jeżeli C_j otrzyma komunikat

$$m = \text{„prepare_to_roll_back * last_sent_msg}_i[j]\text{”}$$

dziedziczy to żądanie pod warunkiem, że:

$$(\text{agree_to_roll_back}_j) \wedge (\text{last_rcvd_msg}_j[i] > \text{last_sent_msg}_i[j]) \wedge \\ (C_j \text{ nie ma jeszcze odziedziczonych dotąd innych żądań do wycofania})$$

Jeśli C_j odziedziczy żądanie, przygotowuje do restartu P_j , rozsyła żądanie do wszystkich kontrolerów C_r ze zbioru roll_cohort_i i oczekuje na odpowiedzi z wartościami $\text{agree_to_roll_back}_j$.

Kiedy wszystkie odpowiedzi nadejdą ustawiana jest własna odpowiedź do kontrolera C_i :

$$\text{agree_to_roll_back}_j = (\text{agree_to_roll_back}_j) \wedge (\bigvee_{C_r \in \text{roll_cohort}_j} \text{agree_to_roll_back}_r) \wedge \\ (\bigvee_{C_r \in \text{roll_cohort}_j} P_r \text{ nie jest uszkodzony})$$

Jeśli C_j nie odziedziczy żądania od C_i wysyła natychmiast odpowiedź:

$$m = \text{„agree_to_roll_back}_j\text{”}.$$

Ażeby wycofać przetwarzanie rozproszone w przypadku wystąpienia błędu inicjator C_β wysyła do kontrolera uszkodzonego procesu C_i komunikat:

$$m = \text{„prepare_to_roll_back * 0”}.$$

Gdy kontroler C_β zbierze wszystkie odpowiedzi z wartościami *true*, decyduje się na wycofanie obliczeń tych wszystkich procesów, w przeciwnym przypadku obliczenia są kontynuowane.

Również w tym algorytmie krytyczne dla poprawności i dla odporności na błędy tego algorytmu jest to, aby, od kiedy C_i otrzyma żądanie do wycofania się, do czasu otrzymania decyzji od inicjatora zabronione jest wysyłanie jakichkolwiek wiadomości przez P_i .

Zachowanie się inicjatora C_β przedstawia blok S7. Akcje podejmowane przez kontroler C_i przedstawiają bloki S2-S3, S8 – S9.

S7: *when an initiator C_β decides to roll back computation due to fail_stop of P_i , C_β will do*

```
send ( $C_\beta$ ,  $C_i$ , „prepare_to_roll_back * 0”);  
rec ( $C_\beta$ ,  $C_i$ , „agree_to_roll_backi”);  
if agree_to_roll_backi  
  then  
    send ( $C_\beta$ ,  $C_i$ , „roll_back”);  
  else  
    send ( $C_\beta$ ,  $C_i$ , „do_not_roll_back”);
```

S8: *when C_i receives from C_j message*

*$m = \text{„prepare_to_roll_back * last_sent_msg}_j[i] \text{”}, C_i$ will do*

```
if (agree_to_roll_backi)  $\wedge$   
  (last_rcvd_msgi[j] > last_sent_msgj[i])  $\wedge$   
  (first_msg_to_roll_back)  
  then  
    /* disable sending messages from  $P_i$  */  
    sendig_enablei;  
    first_msg_to_roll_backi = false;  
    for all  $C_r \in \text{roll\_cohort}_i$  do  
      send ( $C_i$ ,  $C_r$ , „prepare_to_roll_back * last_sent_msgi[r]”);  
    for all  $C_r \in \text{roll\_cohort}_i$  do  
      rec ( $C_i$ ,  $C_r$ , „agree_to_roll_backr”);  
    agree_to_roll_backi = (agree_to_roll_backj)  $\wedge$   
      ( $\bigvee_{C_r \in \text{roll\_cohort}_i}$  agree_to_roll_backr)  $\wedge$   
    send ( $C_i$ ,  $C_j$ , „agree_to_roll_backi = ”);
```

S9: **when** C_i receives from C_j message $m = \text{„roll_back”}$ or $m = \text{„do_not_roll_back”}$,
 C_i will do

if $m = \text{„roll_back”}$

then

for all $C_r \in \text{roll_cohort}_i$ **do**

send ($C_i, C_r, \text{„roll_back”}$);

roll_back_i ;

else

for all $C_r \in \text{roll_cohort}_i$ **do**

send ($C_i, C_r, \text{„do_not_roll_back”}$);

signal(sending_enable_i);

$\text{first_msg_to_roll_back}_i = \text{true}$;

Algorytm Wang – Fuchs

Autorzy algorytmu poczynili następujące założenia odnośnie modelu systemu rozproszonego:

- procesy komunikują się tylko poprzez wymianę komunikatów,
- kanały komunikacyjne są niezawodne i w ogólności nie muszą spełniać reguły FIFO,
- awaria procesu polega na jego zatrzymaniu, zdarzenie to jest wykrywalne przez inne procesy, po czym może nastąpić wycofanie i wystartowanie zatrzymanego procesu,
- awarie w systemie nie powodują rozdzielenia połączeń pomiędzy węzłami,
- przetwarzanie jest niedeterministyczne,
- ze względu na spójność, wszystkie wiadomości są znaczące (tzn. powodują zmianę stanu),
- wszystkie zdarzenia w systemie mogą być wycofane.

Ogólnie, idea algorytmu oparta jest o graf punktów kontrolnych. Graf ten składa się z węzłów reprezentujących punkty kontrolne oraz z krawędzi oznaczających relację zależności pomiędzy poszczególnymi punktami kontrolnymi. K -ty punkt kontrolny procesu P_i zapisujemy jako $ckpt_i^k$, gdzie $k \geq 1$, $1 \leq i \leq n$, n – liczba procesów biorących udział w przetwarzaniu. Aby wprowadzić pojęcie *zależności punktów kontrolnych*, definiuje się tzw. *interwał punktu kontrolnego* określony jako czas pomiędzy dwoma punktami kontrolnymi tego samego procesu. Tak więc, czas pomiędzy punktem kontrolnym $ckpt_i^k$ oraz $ckpt_i^{k+1}$ jest k -tym interwałem punktu kontrolnego procesu P_i lub w skrócie k -tym interwałem tego procesu.

Punkt kontrolny $ckpt_j^l$ jest *bezpośrednio zależny* od punktu kontrolnego $ckpt_i^k$ wtedy i tylko wtedy, gdy:

- $i = j$, oraz $l = k + 1$, lub
- $i \neq j$, i podczas k -tego interwału procesu P_i została wysłana wiadomość m odebrana przez proces P_j w jego $l - 1$ interwale.

Ażeby możliwa była konstrukcja grafu zależności pomiędzy punktami kontrolnymi poszczególnych procesów, do każdej wiadomości dołączane są dodatkowe informacje kontrolne jak również dodatkowe informacje zapisywane są w pamięci stałej wraz z punktami kontrolnymi i logami. Przyjęto następujące nazewnictwo:

- $ckpt_num_i$ - numer interwału procesu P_i ,
- $send_num_i$ - numer sekwencyjny wiadomości wysłanej przez proces P_i ,
- $send_log_i$ - struktura zawierająca pary ($send_num_i, j$), a więc informację: „jaka wiadomość wysłana została do jakiego procesu”,
- rec_log_i - struktura zawierająca czwórki ($m, send_num_j, ckpt_num_j, j$), a więc informacje typu: „odebrano wiadomość m o numerze sekwencyjnym $send_num_j$, wysłaną przez proces P_j podczas jego $ckpt_num_j$ interwału.

Działanie podczas pracy bez awarii.

Podczas normalnej pracy systemu rozproszonego, algorytm kontroluje wymianę komunikatów w sposób następujący:

- do każdej wysyłanej przez proces P_i wiadomości m , kontroler C_i tego procesu dołącza zwiększony o jeden numer sekwencyjny $send_num_i$ oraz aktualny numer interwału $ckpt_num_i$ procesu P_i ,
- kontroler procesu będącego odbiorcą komunikatu m w sposób transparentny dla procesu aplikacyjnego, rozpakowuje i zapamiętuje w opisanych wyżej strukturach danych, przesyłane informacje kontrolne.

Ponadto każdy kontroler spontanicznie i niezależnie od innych kontrolerów może zdecydować o zapisaniu kolejnego punktu kontrolnego procesu P_i . Oprócz samego punktu kontrolnego $ckpt_i^k$ zapisywane są również struktury rec_log_i oraz $send_log_i$ wraz z uaktualnioną $ckpt_num_i$. Po tej operacji struktury rec_log_i oraz $send_log_i$ zostają wyzerowane.

Działanie podczas awarii.

Kiedy kontroler procesu aplikacyjnego wykryje jego awarię, decyduje o rozpoczęciu fazy wycofania i odtwarzania. Ażeby prawidłowo wycofać i wznowić przetwarzanie, kontroler musi skonstruować aktualny graf zależności punktów kontrolnych. W tym

celu rozsyła do wszystkich kontrolerów żądanie o przesłanie znanych im informacji o komunikacji pomiędzy procesami aplikacyjnymi. W odpowiedzi na to żądanie, kontrolery procesów pracujących bez awaryjnie, dokonują zapisów punktów kontrolnych tych procesów, po czym przesyłają zgromadzone dotychczas informacje do kontrolera żądającego. Pozostałe kontrolery odpowiadają przesyłając ostatnie wartości $ckpt_i^k$, rec_log_i , $send_log_i$ oraz $ckpt_num_i$.

Kontroler inicjujący fazę odtwarzania, po zebraniu wszystkich odpowiedzi, tworzy globalny graf zależności punktów kontrolnych. Graf ten rozszerza się o tzw. krawędzie *rollback edges*, które reprezentują wiadomości nie zapisane jeszcze w logu. Tak więc szukając optymalnej linii odtwarzania, węzły z wchodzącymi *rollback edges* muszą zostać odrzucone jak również inne węzły osiągalne z tychże.

Tak zmodyfikowany graf zależności przeglądany jest w celu znalezienia optymalnej linii odtwarzania. Poszukiwanie rozpoczyna się od jądra grafu składającego się z węzłów reprezentujących ostatnie punkty kontrolne poszczególnych procesów. Jeśli istnieje krawędź łącząca dwa węzły z jądra grafu, późniejszy z nich musi zostać zastąpiony w jądrze, wcześniejszym punktem kontrolnym tego samego procesu. Krok ten jest powtarzany do momentu, w którym nie ma żadnej krawędzi łączącej węzły w jądrze grafu. Węzły należące do jądra reprezentują punkty kontrolne tworzące linię odtwarzania. Przetwarzanie jest wycofywane do tej linii odtwarzania i wznowiane. Po wznowieniu, wiadomości *przychodzące* są przetwarzane z logu aż do jego wyczerpania. Wiadomości *wychodzące* wysyłane są ponownie, dlatego potrzebny jest dodatkowy mechanizm odrzucający duplikowane wiadomości.

Algorytm Manetho

Autorami algorytmu Manetho są Elmootazebellah N. Elnozahy i Willy Zwaneepoel. Algorytm ten realizuje techniką adaptacyjnego tworzenia punktu kontrolnego.

Cechy charakterystyczne.

- niski narzut w czasie normalnej pracy dzięki uniknięciu synchronicznego zapisu (pesymistycznego) na trwałym nośniku przez większość czasu;
- ograniczone cofanie (ang. *rollback*) procesów do punktów kontrolnych – cofane są tylko procesy niesprawne i tylko do ostatnich punktów kontrolnych;
- redukcja opóźnienia zatwierdzenia operacji wejścia/wyjścia (ang. *output commit*) przez asynchroniczne wysyłanie komunikatów do świata zewnętrznego (bez koordynacji wielu procesów);
- tolerowanie arbitralnej liczby awarii, wliczając niesprawności, które mogłyby wystąpić podczas odtwarzania.

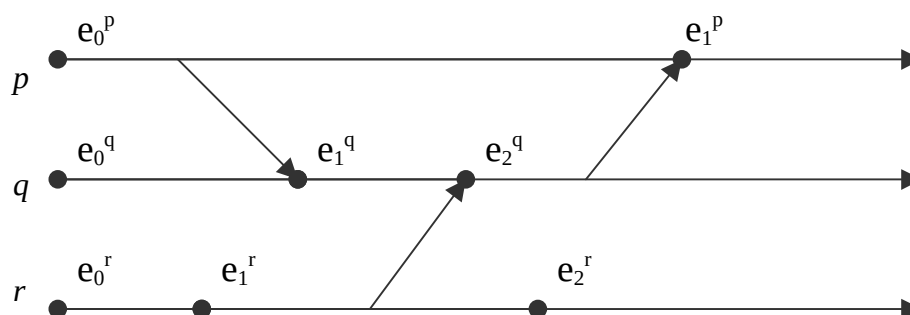
Założenia.

- W przetwarzaniu rozproszonym uczestniczą moduły odtwarzania, zatrzymujące swoją pracę w przypadku niesprawności (ang. *fail stop recovery units*), zwane dalej *RU*;
- *RU* zawierają jeden lub więcej wątków, które zmieniają stan *RU*
- Każdy *RU* ma dostęp do trwałego nośnika;
- Po cofnięciu *RU* może być uruchamiany na dowolnej dostępnej maszynie z grona uczestniczących w przetwarzaniu;
- Praca *RU* składa się z sekwencji deterministycznych zmian stanów, które rozpoczynają się niedeterministycznym zdarzeniem (odebraniem komunikatu, niedeterministycznym zdarzeniem wewnętrznym); zdarzenie i występujące w module odtwarzania $RU p$ oznacza się e_i^p ;
- Komunikacja odbywa się tylko poprzez wymianę komunikatów przez asynchroniczną sieć komputerową;
- Komunikaty mogą być gubione, duplikowane, przesyłane w niewłaściwej kolejności, opóźnione powyżej czasu oczekiwania;

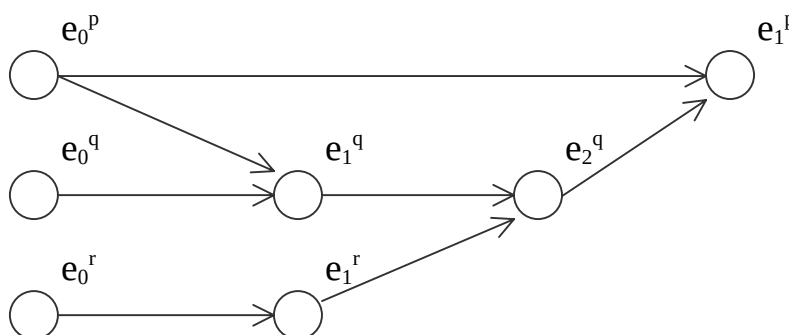
- Sieć w przypadku uszkodzenia nie dzieli się na oddzielne, działające części;
- Dane odebrane przez RU z urządzenia wejścia/wyjścia muszą być zapisane na trwałym nośniku zanim RU wyśle komunikat do innego RU lub urządzenia wejścia/wyjścia

Opis algorytmu.

W algorytmie wykorzystywany jest graf poprzedzania (ang. *antecedence graph*) stąd w opisie algorytmu opis tego grafu. Graf poprzedzania zdarzenia e_i^p , $AG(e_i^p)$, jest grafem skierowanym acyklicznym. Zawiera węzeł reprezentujący zdarzenie e_i^p oraz węzły reprezentujące wszystkie zdarzenia, które poprzedzają e_i^p . W wyniku zajścia zdarzenia odebrania komunikatu odpowiedni węzeł posiada dwie krawędzie wejściowe: z węzła reprezentującego poprzednie zdarzenie w tym samym module kontrolnym oraz z węzła reprezentującego zdarzenie nadania tego komunikatu. Należy przy tym nadmienić, że w tym przypadku zdarzenie nadania komunikatu jest traktowane jako zapoczątkowane przez jakieś zdarzenie wewnętrzne. Węzeł reprezentujący niedeterministyczne zdarzenie wewnętrzne posiada tylko jedną krawędź – z węzła reprezentującego poprzednie zdarzenie w tym samym module kontrolnym. Przykład grafu poprzedzania dla przykładowego przetwarzania rozproszonego przedstawiają rysunki:



Rysunek 8 Przykładowe przetwarzanie rozproszone



Rysunek 9 Przykładowy graf poprzedzania

Wszystkie węzły w grafie poprzedzania posiadają dodatkowe pola, zawierające następujące informacje: typ (zdarzenie odebrania komunikatu lub zdarzenie wewnętrzne), identyfikator odbiorcy komunikatu, identyfikator nadawcy, indeks zdarzenia, unikalny identyfikator komunikatu. Natomiast nie zawierają kopii danych komunikatu. Graf poprzedzania $AG(e_i^p)$ zawiera więc informacje użyteczne przy powtarzaniu zdarzeń podczas wstecznego odtwarzania stanu obliczeń.

W czasie normalnego wykonywania programu rozproszonego każdy RU utrzymuje w pamięci ulotnej graf AG bieżącego zdarzenia oraz zapis historii komunikatów, tzn. treść oraz nadawcę każdego odebranego komunikatu. W przypadku nadania komunikatu RU razem z komunikatem przesyła graf AG bieżącego stanu. W celu zmniejszenia ilości przesyłanej informacji, a więc zwiększenia efektywności stosuje się przesłanie tylko pewnych części grafu (ang. *incremental piggybacking*). Z definicji $AG(e_i^p)$ jest poprawnym podgrafem $AG(e_{i+1}^p)$. Wszystkie RU q , które komunikują się z p z każdym komunikatem wysyłanym do p przesyłają tylko maksymalny indeks j , reprezentujący stan modułu q we własnym grafie poprzedzania. Gdy natomiast p przesyła komunikat do modułu q , będąc w stanie e_i^p , wraz z komunikatem przesyła tylko $AG(e_i^p) - AG(e_j^p)$. Odbiorca komunikatu odnotowuje nowe zdarzenie i konstruuje nowy graf AG korzystając z poprzedniego oraz z informacji dostarczonych z komunikatem.

Periodycznie każdy moduł RU zapisuje swój punkt kontrolny w pamięci nieulotnej. Nie ma przy tym żadnej koordynacji zapisów punktów kontrolnych z innymi modułami. Po zapisaniu punktu kontrolnego w pamięci trwałej zapisywane są również historia komunikatów oraz graf poprzedzania. Okazyjnie każdy RU zapisuje w pamięci trwałej graf poprzedzania bieżącego stanu. Od tej chwili podgraf zapisany na trwałym nośniku nie powinien być przesyłany z komunikatami. Graf poprzedzania jest również zapisywany w pamięci trwałej przed każdym wysłaniem komunikatu do świata zewnętrznego (również bez koordynacji z innymi modułami).

Z uwagi na niedeterministyczny czas przesyłania komunikatów może zdarzyć się sytuacja, w której RU q otrzyma komunikat od RU p po jego awarii i cofnięciu. Nie ma wtedy możliwości stwierdzenia, czy komunikat pochodzi od modułu sprzed awarii czy po cofnięciu. Wprowadza się więc numer inkarnacji (ang. *incarnation number*), który jest przesyłany z każdym komunikatem. Po ewentualnej awarii i cofnięciu modułu zwiększa on numer inkarnacji i informuje o tym wszystkie inne moduły. Komunikaty oznaczone nieaktualnym numerem inkarnacji są odrzucane.

Algorytm odtwarzania stanu modułu $RU\ p$ rozpoczyna się od odtworzenia stanu (z punktu kontrolnego), historii komunikatów, numeru inkarnacji oraz grafu poprzedzania AG z pamięci trwałej. Numer inkarnacji $INCNUM$ jest zwiększany o jeden i zapisywany niezwłocznie w pamięci trwałej. W tablicy numerów inkarnacji $INCVEC$ na pozycji p zapisywany jest bieżący numer inkarnacji. Zmienna G (typu graf poprzedzania) jest inicjowana odtworzonym grafem AG . Następnie do wszystkich pozostałych modułów wysyłane jest żądanie przesłania ich numerów inkarnacji oraz grafów poprzedzania $AG(e_k^p)$ takich, że k jest maksymalnym indeksem zdarzenia odebrania komunikatu od p w grafie modułu q . Po otrzymaniu żądania od p każdy moduł q zapisuje swój graf poprzedzania na stałym nośniku, szuka k , do tablicy numerów inkarnacji odrzucanych komunikatów $REJECTVEC$ na pozycji p zapisuje k i wysyła żądane dane. Wszystkie moduły q odrzucają komunikaty zawierające graf AG , w których istnieje węzeł reprezentujący stan o indeksie większym od k . Graf G jest sukcesywnie uzupełniany o przysłane grafy, a w tablicy $INCVEC$ są zapisywane kolejne przysłane numery inkarnacji na pozycjach wskazujących na nadawcę. Kiedy wszystkie potrzebne dane dotrą do odtwarzanego modułu, p rozsyła do wszystkich innych modułów tablicę $INCVEC$. Po otrzymaniu tych danych q uaktualnia swoją tablicę $INCVEC$, wybierając większą spośród odpowiadających sobie wartości tablicy własnej i nadesłanej. Następnie unieważniana jest liczba zapisana na pozycji p w tablicy $REJECTVEC$, a tym samym odrzucenie komunikatów związanych z tym modułem. Moduł odtwarzany p wykonuje następnie odpowiednie zdarzenia według otrzymanego grafu G , aby osiągnąć stan odpowiadający zdarzeniu z maksymalnym indeksem. W przypadku, gdy odtwarzanym zdarzeniem jest odebranie komunikatu, p żąda danych tego komunikatu z historii komunikatów nadawcy, w przeciwnym przypadku zdarzenie jest wykonywane od razu. Należy zauważyć, że wszystkie komunikaty używane do komunikacji z innymi modułami w czasie odtwarzania nie są rejestrowane tak jak komunikaty przetwarzania. Podczas odtwarzania wszystkie komunikaty związane z urządzeniami wejścia/wyjścia nie są wysyłane, ale zapisywane w pamięci ulotnej.

Struktury danych.

N	- liczba modułów programu rozproszonego;
c	- indeks bieżącego stanu odtwarzanego modułu odczytany z pamięci trwałej;
$INCNUM$	- liczba inkarnacji;

- INCVEC* - tablica liczb inkarnacji wszystkich modułów (o wymiarze N);
- REJECTVEC* - tablica liczb inkarnacji odrzucanych komunikatów dla każdego modułu (o wymiarze N);
- G - odtwarzany graf;
- STATEINDEX* - odtwarzany indeks stanu;

Algorytm odtwarzania.

Initial state at all modules q :

$INCNUM := 1;$
 $INCVEC[1..N] := 1;$
 $REJECTVEC[1..N] := \infty;$

At recovered module p :

$c :=$ current state index;
 $INCNUM := INCNUM + 1;$
save $INCNUM$ on stable storage;
 $INCVEC[p] := INCNUM;$
 $G := AG(e_c^p);$

for all modules $q \neq p$ do

send $Get_AG(p)$;

Upon receiving message $Send_AG(INQ, AGQ)$

$G := G \cup AGQ;$

$INCVEC[q] := INQ;$

for all modules $q \neq p$ do

send $Send_INC(p, INCVEC)$ message;

$m := \max j \text{ such that } e_j^p \in G;$

$STATEINDEX := c;$

while $STATEINDEX \leq m$ **do**

 execute up to next event without sending application messages;

$STATEINDEX := STATEINDEX + 1;$

if next event is a receive **then**

 request message from sender's log;

else re-execute internal event;

At all modules q

 Upon receiving message $Get_AG(p)$

save AG **on stable storage;**

$k := \max j \text{ such that } e_j^p \in G;$

$REJECTVEC[p] := k;$

Send $SendAG(INCNUM, AG(e_c^p))$ **message;**

 Upon receiving message $Send_INC(p, PINCVEC)$

for all $i := 1..N$ **do**

$INCVEC[i] := \max(INCVEC[i], PINCVEC[i]);$

$REJECTVEC[p] := \infty;$